

Natural Language Processing Lab

CASE STUDY

Title : Implement Dependency Parsing using Spacy and visualize syntactic relationships

Unit II : Syntax (CO2)

Name: S. Gayathri

Roll. No: 23B81A7219

Branch: AID

1. Problem Statement and Objectives

Dependency parsing is an NLP technique used to identify the grammatical relationships between words in a sentence. In this project, spaCy is used to analyze English sentences and identify the dependency relation, head word, and part-of-speech (POS) of each word.

The main objectives are:

To perform dependency parsing using spaCy.

To identify the syntactic relationship between words.

To display the dependency, head word, and POS of each token.

To visualize the dependency tree using DisplaCy.

2. Dataset Description and Source

The dataset used for this project is a custom English sentence dataset containing 100 sentences. The sentences cover simple everyday topics such as students, teachers, animals, family, sports, transportation, and workplaces.

The dataset is stored in a text file named document.txt, which is directly compatible with the given Python program.

Dataset details:

File name: document.txt

Number of sentences: 100

Language: English

Format: Plain text (.txt)

Source: Custom-created English sentence corpus for dependency parsing

The sentences are intentionally simple so that the dependency relationships can be easily understood and visualized.

3. Tools and Libraries Used

The following tools and libraries are used:

Python: Used to write and execute the program.

spaCy: Used for NLP processing and dependency parsing.

en_core_web_sm: Pre-trained English language model used by spaCy.

DisplaCy: Used to visualize the dependency relationships as a tree.

4. System Architecture / Workflow

The overall workflow of the project is:

Input Text (file1.txt)



Read Text



spaCy NLP Pipeline



Sentence & Token Processing



Dependency Parsing



Dependency + Head + POS Output



DisplaCy Visualization

The text is first read from file1.txt. It is then passed to the spaCy model, which analyzes the sentences and identifies the syntactic relationships between words. The results are displayed in the terminal and visually using DisplaCy.

5. Text Preprocessing Steps

The input dataset contains plain English sentences. The text is read from document.txt using Python file handling.

spaCy automatically performs the required linguistic processing, including:

Sentence segmentation

Tokenization

Part-of-speech tagging

Dependency analysis

For this project, stopword removal, stemming, and lemmatization are not performed because the purpose is to preserve the original sentence structure for dependency parsing.

6. Algorithm/Model Implementation

Source Code:

```
import spacy

from spacy import displacy

nlp = spacy.load("en_core_web_sm")

f1=open("file1.txt","r")

text=f1.read()

doc = nlp(text)

print("Dependency Parsing Results\n")

for sent in doc.sents:

    print("\nSentence:", sent.text)

    print("-" * 80)

    for t in sent:

        print(

            f"{t.text:<20}"

            f"{t.dep_:<15}"

            f"{t.head.text:<20}"

            f"{t.pos_:<10}"

        )

    print("\nStarting dependency visualization...")

    displacy.render(doc, style="dep",jupyter=True)
```

The project uses the spaCy en_core_web_sm pre-trained English model.

The model analyzes each sentence and assigns a dependency relationship to every token. The program uses the following spaCy attributes:

t.text – Displays the word/token.

t.dep_ – Displays the dependency relationship.

t.head.text – Displays the syntactic head of the word.

t.pos_ – Displays the part-of-speech tag.

The program also uses:

```
displacy.serve(doc, style="dep")
```

to generate a visual dependency tree.

7. Experimental Results and Evaluation

The program successfully processed all 100 sentences from document.txt. For every sentence, spaCy identified the tokens, their dependency relations, head words, and part-of-speech (POS) tags.

For example, the dataset contains the sentence:

“The cat sits on the mat.”

The generated dependency output is:

Token	Dependency	Head	POS
The	det	cat	DET
cat	nsubj	sits	NOUN
sits	ROOT	sits	VERB
on	prep	sits	ADP
the	det	mat	DET
mat	pobj	on	NOUN
.	punct	sits	PUNCT

From this result, “sits” is identified as the root of the sentence. The word “cat” is the subject of “sits”, while “mat” is connected to “on”. The words “The” and “the” are identified as determiners.

The program generated similar dependency information for all 100 sentences. The POS output also shows that the model correctly identifies common categories such as NOUN, VERB, DET, ADP, ADJ, ADV, AUX, PRON, PROPN, and PUNCT.

8. Sample Outputs / Screenshots

The program produces two main outputs.

Terminal Output:

Dependency Parsing Results

Sentence: The cat sits on the mat.

...

Starting dependency visualization...

The terminal shows each token, its dependency relation, head word, and POS tag.

Visual Output:

After the message:

Starting dependency visualization...

DisplaCy opens the dependency visualization in a browser. The words are connected using arrows that show their syntactic relationships.

Screenshots to include in the final submission:

Screenshot of the terminal showing the dependency parsing results.

Screenshot of the DisplaCy dependency tree in the browser.

9. Conclusion and Future Scope

This project demonstrates how dependency parsing can be performed using spaCy. The system successfully analyzes 75 English sentences and identifies the syntactic relationships between their words. The textual output provides detailed token information, while DisplaCy makes the dependency structure easier to understand visually.

In the future, the project can be improved by using a larger real-world annotated dataset and calculating UAS and LAS for numerical evaluation. It can also be extended to applications such as information extraction, question answering, text summarization, and natural language understanding.

